

Fly GACA — Phase 2 Multi-Repo Agent Architecture

SECTION 06-operations-it TYPE architecture STATUS active OWNER Founder

UPDATED 2026-08-26

Version: 2.0.0 | Effective: 2026-08-26 | Duration: Weeks 3–10 (after Phase 5 Week 1–2)

Purpose. Migrate the flat agent roster into a wshobson plugin architecture with unified MCP state management, enabling coordinated multi-repo workflows, qualified harness deployment (Claude Code, Cursor, desktop app), and commercial-quality multi-agent orchestration.

Framework Selection

Research Complete

Both `wshobson/agents` and `msitarzewski/agency-agents` offer distinct strengths:

Framework	Best For	Advantage
wshobson/agents	Multi-repo orchestration	93 plugins, 202 agents, single source of truth, 16 orchestrators, enterprise quality gates
msitarzewski/agency-agents	Personality + UX	5.5K stars, persona-driven agents, MCP memory patterns, 5 full workflow examples, desktop app

Decision: Hybrid Approach

- **Backbone:** wshobson/agents for plugin architecture, multi-harness adapters, orchestrators, quality gates
- **Personality Layer:** Inspired by msitarzewski emoji + color + vibe model
- **State Management:** MCP servers (proven by msitarzewski examples)
- **Team Experience:** Start wshobson for operational rigor; layer msitarzewski personas for human collaboration

Plugin Architecture

Four Plugins (Fly GACA Family)

[!IMPORTANT] Names below are the plan's, not the repository's. Phase 2.2 reconciled this architecture onto the plugins that already shipped rather than building a second set beside them. The real roster is the one in `.claude-plugin/marketplace.json` :

Planned name	Actual plugin	Lives in
office-governance	office-governance (+ office-docs for the doc commands)	ay2m/Office
product-engineering	flygaca-product	ay2m/FlyGACA
flight-service	captain-adel-service	ay2m/Captain-Adel
fly-gaca-operations	family-orchestrators	ay2m/Office

The three placeholder directories this document originally described in `ay2m/Office` held a manifest and nothing else, in a `.claude-plugin.json` shape Claude Code does not read. They were removed in Phase 2.2.

Plugin	Owner	Agents	Purpose
office-governance	ay2m/Office	7	Document authoring, compliance, entity facts, cross-repo sync
product-engineering	ay2m/FlyGACA	5	React, Express, corpus, SQL, RAG
flight-service	ay2m/Captain-Adel	4	Curriculum, model tuning, learner data, deployment
fly-gaca-operations	ay2m/Office	5 orchestrators	Cross-repo workflows (full-sync, feature-ship, compliance-audit, security-hardening, performance-sprint)

Office Governance Plugin (7 agents)

Agents:

- **doc-smith** (cyan) — Markdown/HTML → PDF pipeline, front-matter, templates
- **ar-mirror** (purple) — Arabic translation, `ar/` mirror maintenance
- **ksa-compliance** (yellow) — PDPL, ZATCA, MISA scaffolding
- **family-warden** (magenta) — Contract parity, entity-facts consistency
- **governance-auditor** (blue) — Decision log, policy drift detection

- **entity-facts-guardian** (green) — Entity facts, IBAN protection, versioning
- **cross-repo-sync** (orange) — Office ↔ FlyGACA ↔ Captain-Adel coordination

Skills:

- office-ci-gates — Front-matter, PDF coverage, staleness
- entity-facts-validation — Entity facts parity
- cross-repo-stamping — Contract versioning and stamping

Commands:

- office-sync — Manual cross-repo synchronization

Product Engineering Plugin (5 agents, ay2m/FlyGACA)

Agents:

- **react-19-architect** (teal) — React 19, Vite, RTL, TypeScript strict
- **express-backend-pro** (indigo) — Express 5, Cloud Run, security
- **regulatory-corpus-keeper** (amber) — GACAR corpus, citation tiers
- **sql-migrator** (stone) — PostgreSQL, forward-only migrations
- **genkit-rag-specialist** (lime) — Gemini, RAG pipeline, grounding

Skills:

- react-typescript-strict
- express-security-patterns
- gacar-corpus-policy
- postgresql-migrations
- gemini-rag-patterns

Commands:

- feature-launch — React + Express + corpus coordination
- security-hardening — Security audit across frontend + backend
- performance-sprint — Bundle + endpoint + query optimization

Flight Service Plugin (4 agents, ay2m/Captain-Adel)

Agents:

- **flight-curriculum-designer** (sky) — Syllabus, exam scope, learner paths
- **ml-instructor-trainer** (violet) — Persona tuning, model fine-tuning, eval
- **flight-data-pipeline-engineer** (rose) — Learner data, flight-hours, PDPL
- **instructor-deployment-steward** (fuchsia) — Cloud Run hosting, webhooks

Skills:

- aviation-pedagogy

- flight-instructor-personas
- pdpl-learner-data
- captadel-deployment

Commands:

- instructor-launch — Curriculum + model + deployment sequencing
- personalization-tuning — Learner feedback loop → model improvement

Operations & Orchestration Plugin (5 orchestrators, ay2m/Office)

Agents:

- **operations-orchestrator** (rose) — Multi-agent workflow coordination

Orchestrators:

Workflow	Agents	Purpose
<code>/full-sync</code>	cross-repo-sync, governance-auditor, entity-facts-guardian + product-engineering, flight-service	Morning check-in: Office ↔ product ↔ service; verify contracts, entity facts, decision log
<code>/feature-ship</code>	react-19-architect, express-backend-pro, flight-curriculum-designer, cross-repo-sync	Ship a feature: React → Express API → Captain Adel integration → Office docs
<code>/compliance-audit</code>	ksa-compliance, entity-facts-guardian, flight-data-pipeline-engineer, governance-auditor	Quarterly: PDPL, ZATCA, Nitaqat, decision log review
<code>/security-hardening</code>	(product-engineering security flow), (flight-service data security), ksa-compliance	Security review: XSS, injection, data residency, PDPL
<code>/performance-sprint</code>	(product-engineering performance flow), sql-migrator, flight-data-pipeline-engineer	Performance: bundle, endpoints, queries, learner throughput

MCP Memory State Management

Shared MCP server stores context across agent sessions:

Memory Key	Agents Writing	Purpose
<code>office-entity-facts-v1</code>	entity-facts-guardian	Current entity facts snapshot from company-facts.md
<code>office-governance-v1</code>	governance-auditor	Decision log state and policy drift
<code>office-contract-v1</code>	family-warden	Family contract version, sha, parity status
<code>product-architecture-v1</code>	react-19-architect, express-backend-pro	Frontend + backend design decisions

Memory Key	Agents Writing	Purpose
corpus-index-v1	regulatory-corpus-keeper	GACAR document index, freshness timestamp
captain-adel-model-v1	ml-instructor-trainer	Model version, persona tuning params
flight-data-v1	flight-data-pipeline-engineer	Learner data ingestion status, currency checks
cross-repo-health-v1	cross-repo-sync, family-warden	Contract parity check, sha verification

Deployment Strategy

Harnesses (Multi-Harness Deployment)

Agents deployable to:

- Claude Code** (native CLAUDE.md integration, marketplace discovery)
 - Full tool access, all agents
 - Marketplace-hosted plugin discovery
- Cursor** (via wshobson adapter framework, auto-generated rules)
 - Subset of agents (adapter guards tool access)
 - Rules-based agent invocation
- Desktop App** (optional, msitarzewski-inspired)
 - Browse agent roster visually
 - Lightweight GUI for non-technical team members
 - MCP memory visualization

Plugin Registry

- Plugins registered in `.claude/plugins/marketplace.json`
- `flygaca-family` collection: three active plugins (office-governance, product-engineering, flight-service) + orchestration
- Auto-discovered by Claude Code when collection is installed
- Upstream contribution: aviation-specific agents (regulatory-corpus-keeper, flight-curriculum-designer, ml-instructor-trainer) to wshobson marketplace post-launch

Integration with Phase 5

Phase 5 (Weeks 1–12, parallel to Phase 2 Weeks 3–10):

- **Phase 5 Week 1 kickoff** → Agent champions assigned, first invocations logged
- **Phase 2 Weeks 3–10 build** → Agents tested in daily workflows
- **Phase 5 Weeks 2–12 metrics** → Invocation log feeds Phase 2 effectiveness validation
- **Phase 5 Week 12 review** → Agent portfolio readiness for Phase 6

Agents are scaffolding; Phase 5 measures whether they actually add ROI. Phase 2 architecture supports that measurement with MCP memory state and effectiveness scoring formulas.

Implementation Phases (5 phases, Weeks 3–10)

Phase 1: Foundation (Weeks 3–4)

1. Create wshobson plugin structure (`plugins/` directory)
2. Port existing agents into `office-governance` plugin
3. Add 3 new Office agents (already created: governance-auditor, entity-facts-guardian, cross-repo-sync)
4. Set up MCP memory server (standard MCP architecture)
5. CI: Extend `.claude/agents/` validation to plugin structure

Deliverables:

- `.claude/plugins/office-governance/` complete (7 agents, 3 skills, 1 command)
- MCP memory server skeleton
- Updated CI gates for plugin structure

Phase 2: Product Engineering Agents (Weeks 5–6) — delivered, reconciled

What this phase planned. A new `product-engineering` plugin in `ay2m/FlyGACA` carrying five agents (`react-19-architect` , `express-backend-pro` , `regulatory-corpus-keeper` , `sql-migrator` , `genkit-rag-specialist`), five skills and three commands.

What was already there. `ay2m/FlyGACA` had shipped a `flygaca-product` plugin covering exactly those five domains — `react-surface` , `express-api` , `corpus-pipeline` , `sql-schema` , `rag-grounding` — in the correct `.claude-plugin/plugin.json` format, registered in the family marketplace, with agent content grounded in the repo's real paths and gates. Building the planned plugin alongside it would have given one repo **ten agents for five jobs**, which is precisely what success criterion #2 ("No Duplication") forbids.

What was done instead. `flygaca-product` was treated as this phase's deliverable and completed:

Planned	Resolution
5 agents	Already present under repo-native names; canonical ↔ plugin names mapped in <code>ay2m/FlyGACA/CLAUDE.md</code>

Planned	Resolution
5 skills	Added to <code>.claude/plugins/flygaca-product/skills/</code> — <code>react-typescript-strict</code> , <code>express-security-patterns</code> , <code>gacar-corpus-policy</code> , <code>postgresql-migrations</code> , <code>gemini-rag-patterns</code>
<code>feature-launch</code> command	Added — sequences schema → API → corpus → surface with the right gate at each step
<code>security-hardening</code> , <code>performance-sprint</code> commands	Not duplicated. They are family-level and stay in Office's <code>family-orchestrators</code> ; a security or performance pass that stops at one repo's boundary is not the pass those commands promise
Link to FlyGACA CLAUDE.md	Done , and corrected — see below

The split held to is: **skills carry procedures, agents carry role context.** Where both touch a subject the skill states the steps and defers the "why it is shaped this way" to the named agent.

Three defects this phase found and fixed:

1. `tools/agents/check-agents.mjs` **validated nothing in the plugins.** It scanned `plugins/` at the repo root, a path that stopped existing when the tree moved to `.claude/plugins/` in `08147d4`. It reported "all valid" over an empty set. Repointed, scoped to `<plugin>/agents/` (its recursive walk would otherwise have graded commands and skills against the agent schema), and given a **parity check**: a plugin agent that shadows a repo agent must be byte-identical to it. The workflow's `plugins/*/agents/**` path filter was stale in the same way and now reads `.claude/plugins/**`.
2. **Both product repos claimed their agents arrive "via the family contract."** The contract carries entity facts, the chat contract and the repo roster — it distributes no agents. Both `CLAUDE.md` files now name the three real routes: repo-local `.claude/agents/`, the repo's marketplace plugin, or the Office checkout.
3. **The three-tier corpus policy is not a mechanism.** `HOST_SAFE_CORE` / `HOST_ORIGINAL` / `DO_NOT_HOST` appear nowhere in FlyGACA's code or corpus data. They are recorded as stated editorial policy, not an enforced field. AIRAC was restated to what `src/calc/airac.ts` actually implements: 28-day cycles anchored to AIRAC 2001, a 7-day due window — not a "35-day threshold".

Deliverables (actual):

- `ay2m/FlyGACA` : `.claude/plugins/flygaca-product/` at v0.2.0 — 5 agents, **5 skills, 5 commands**
- `ay2m/Office` : three content-free stub plugins removed; `office-governance` converted to the real plugin manifest format and registered in the marketplace; agent gate repaired
- `ay2m/Captain-Adel` : `CLAUDE.md` agent-provenance correction

Not delivered: "validate with one live feature shipping." No feature was shipped through `/feature-launch` — the command is written but unexercised, and should not be described as tested end-to-end until it has driven a real change.

Phase 3: Flight Service Agents (Weeks 7–8)

1. Create `flight-service` plugin structure in ay2m/Captain-Adel
2. Port 4 agents (flight-curriculum-designer, ml-instructor-trainer, flight-data-pipeline-engineer, instructor-deployment-steward)
3. Link to Captain-Adel repo CLAUDE.md
4. Create instructor-launch orchestrator
5. Validate with one curriculum iteration

Deliverables:

- ay2m/Captain-Adel: `.claude/plugins/flight-service/`
- 4 agents with persona
- instructor-launch orchestrator tested

Phase 4: Orchestration + Testing (Week 9)

1. Build `fly-gaca-operations` plugin with 5 orchestrators (full-sync, feature-ship, compliance-audit, security-hardening, performance-sprint)
2. Run full-sync weekly; validate Office ↔ product ↔ service parity
3. Document 3 common workflows (feature ship, compliance audit, performance sprint)
4. Team onboarding: show how to use desktop app (optional) + CLI commands

Deliverables:

- `.claude/plugins/fly-gaca-operations/` with 5 orchestrators
- Workflow documentation (feature-ship, compliance-audit, security-hardening, performance-sprint)
- Team onboarding guide

Phase 5: Production (Week 10+)

1. Deploy to Claude Code + Cursor + optional desktop app
2. Team uses orchestrators in daily workflows
3. Monthly agent ecosystem review (add agents, refine, archive unused)
4. Contribution: upstream aviation-specific agents to wshobson marketplace
5. Metrics integration: Phase 5 invocation log feeds agent effectiveness scoring

Deliverables:

- All plugins deployed and discoverable
- Team using orchestrators daily
- Phase 5 metrics dashboard tracking invocations
- Upstream contributions filed

Success Criteria

1. **Unified Team** — All 16+ agents discoverable in one place; clear ownership per repo
2. **No Duplication** — Office facts, product architecture, service state maintained once; agents read from MCP memory
3. **Orchestration Works** — full-sync runs weekly, zero manual intervention; reports parity
4. **Developer Experience** — Each team uses agents daily without friction
5. **Quality Gates** — Every agent has clear non-inferable facts; decision log records why each earned its slot
6. **Multi-Harness Ready** — Agents deployable to Claude Code, Cursor, optional desktop app (wshobson adapters)

Risks & Mitigations

Risk	Mitigation
Plugin scope creep — 16 agents feels small; pressure to add more	Document "earn its slot" test; every new agent needs non-inferable facts + design review
MCP memory drift — Office facts and product architecture diverge	cross-repo-sync orchestrator runs weekly; governance-auditor flags inconsistencies
Harness compatibility — wshobson plugins work in Claude Code; unclear if all agents work in Cursor/OpenCode	Test with adapter framework in Phase 2.2; may need lite personas for unsupported tools
Personality vs. rigor — msitarzewski vibe feels lightweight vs. wshobson enterprise rigor	Hybrid: wshobson architecture for consistency, msitarzewski personas for approachability
Orchestrator complexity — 5 multi-agent orchestrators could become hard to maintain	Document each as single Markdown command; treat as infrastructure, not agent

Coordination with Phase 5

Timeline alignment:

- Phase 5 Week 1 (Aug 26–Sep 1): Kickoff, agent champions assigned
- Phase 2 Weeks 1–2 (Sep 2–15): Foundation, office-governance plugin built
- Phase 5 Weeks 2–4: Team logs first invocations, effectiveness metrics start
- Phase 2 Weeks 3–4 (Sep 16–29): Product engineering agents ported
- Phase 2 Weeks 5–6 (Sep 30–Oct 13): Flight service agents ported
- Phase 5 Weeks 5–8: Team adoption ramps; agents tested in daily workflows
- Phase 2 Weeks 7–8 (Oct 14–27): Orchestration layer complete, full-sync live
- Phase 5 Weeks 9–12: Metrics deep-dive, effectiveness review, agent retirements decided

- Phase 2 Week 9+ (Oct 28+): Production deployment, upstream contributions

Phase 5 effectiveness scoring informs Phase 2 orchestrator refinement — if an agent is low-ROI in invocation logs, its role in workflows is revisited.

Fly GACA / Phase 2: Multi-Repo Agent Architecture / Phase 2.1 Foundation complete · Phase 2.2 delivered and reconciled